

SKILL WEEK-12

1 . To execute large pipelines, launch multiple jobs, measure system stability, monitor resource usage, detect failures, and analyze performance.

Aim : To implement a C program that creates multiple child processes, executes pipeline commands, monitors their execution, detects failures, and measures the overall execution time.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>

int main()
{
    int pipe1[2];
    int pipe2[2];

    pid_t p1, p2, p3;

    struct timeval start, end;
    gettimeofday(&start, NULL);

    printf("Large Pipeline Execution\n");
    printf("-----\n");

    /* Create pipes */
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* First job: ls */
    p1 = fork();
    if (p1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(pipe1[1], STDOUT_FILENO);
        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", "-l", NULL);
    }
}
```

```
GNU nano 2.2

    perror("exec ls");
    exit(1);
}

/* Second job: grep */
p2 = fork();
if (p2 < 0)
{
    perror("fork");
    return 1;
}

if (p2 == 0)
{
    dup2(pipe1[0], STDIN_FILENO);
    dup2(pipe2[1], STDOUT_FILENO);

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    execlp("grep", "grep", ".c", NULL);
    perror("exec grep");
    exit(1);
}

/* Third job: sort */
p3 = fork();
if (p3 < 0)
{
    perror("fork");
    return 1;
}

if (p3 == 0)
{
    dup2(pipe2[0], STDIN_FILENO);

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);
}
```

```
GNU nano 2.2

/* Parent closes pipes */
close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

printf("Three pipeline jobs launched.\n");
printf("Job 1: ls -l\n");
printf("Job 2: grep .c\n");
printf("Job 3: sort\n\n");

/* Wait for all jobs */
int status1, status2, status3;

waitpid(p1, &status1, 0);
waitpid(p2, &status2, 0);
waitpid(p3, &status3, 0);

gettimeofday(&end, NULL);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_usec - start.tv_usec) / 1000000.0;

printf("\nPipeline execution completed.\n");
if (WIFEXITED(status1) && WEXITSTATUS(status1) == 0)
    printf("Job 1: SUCCESS\n");
else
    printf("Job 1: FAILED\n");

if (WIFEXITED(status2) && WEXITSTATUS(status2) == 0)
    printf("Job 2: SUCCESS\n");
else
    printf("Job 2: FAILED\n");

if (WIFEXITED(status3) && WEXITSTATUS(status3) == 0)
    printf("Job 3: SUCCESS\n");
else
    printf("Job 3: FAILED\n");

printf("\nTotal execution time: %.6f seconds\n", elapsed);

return 0;
}
```

Output :

```

jhanasri@LAPTOP-1R1N8E3:~/OSSP$ cd ~/OSSP
jhanasri@LAPTOP-1R1N8E3:~/OSSP$ nano task_large_pipeline.c
jhanasri@LAPTOP-1R1N8E3:~/OSSP$ gcc -Wall -Wextra task_large_pipeline.c -o task_large_pipeline
jhanasri@LAPTOP-1R1N8E3:~/OSSP$ ./task_large_pipeline

```

Large Pipeline Execution

=====

Three pipeline jobs launched.

Job 1: ls -l

Job 2: grep .c

Job 3: sort

```

-rw-r--r-- 1 jhanasri jhanasri      77 Sep 22 14:17 expected_report.txt
-rw-r--r-- 1 jhanasri jhanasri     130 Sep 22 13:39 calculator.h
-rw-r--r-- 1 jhanasri jhanasri     169 Sep 22 13:41 calculator.c
-rw-r--r-- 1 jhanasri jhanasri     355 Sep 22 14:14 calculator_test.c
-rw-r--r-- 1 jhanasri jhanasri     420 Aug 13 11:45 main_loop.c
-rw-r--r-- 1 jhanasri jhanasri     443 Sep 22 14:07 task_valgrind_leak.c
-rw-r--r-- 1 jhanasri jhanasri     465 Sep 22 13:42 main_module.c
-rw-r--r-- 1 jhanasri jhanasri     662 Aug 17 13:53 week11_T2.c
-rw-r--r-- 1 jhanasri jhanasri     726 Sep 18 03:58 signal_handler.c
-rw-r--r-- 1 jhanasri jhanasri     769 Sep 22 14:03 task_valgrind.c
-rw-r--r-- 1 jhanasri jhanasri     900 Aug 17 06:02 task3_fork.c
-rw-r--r-- 1 jhanasri jhanasri     954 Aug 17 13:32 task1_states.c
-rw-r--r-- 1 jhanasri jhanasri    1018 Sep 18 03:50 client.c
-rw-r--r-- 1 jhanasri jhanasri    1275 Sep 18 03:48 server.c
-rw-r--r-- 1 jhanasri jhanasri    1376 Sep 18 03:32 ls_grep.c
-rw-r--r-- 1 jhanasri jhanasri    1464 Sep 22 13:44 calculator.o
-rw-r--r-- 1 jhanasri jhanasri    1541 Sep 22 13:50 task_error_handling.c
-rw-r--r-- 1 jhanasri jhanasri    1550 Sep 21 04:56 task_signal_actions.c
-rw-r--r-- 1 jhanasri jhanasri    1617 Sep 21 05:54 task_signal_behavior.c
-rw-r--r-- 1 jhanasri jhanasri    1661 Aug  7 08:00 task3_input.c
-rw-r--r-- 1 jhanasri jhanasri    1677 Sep 18 03:26 producer_consumer.c
-rw-r--r-- 1 jhanasri jhanasri    1705 Sep 18 04:13 task2_single_quotes.c
-rw-r--r-- 1 jhanasri jhanasri    1851 Aug 17 13:42 task1_wait.c
-rw-r--r-- 1 jhanasri jhanasri    1851 Aug 17 13:46 task1_wait.c.save
-rw-r--r-- 1 jhanasri jhanasri    1959 Sep 21 05:15 task_signal_forward.c
-rw-r--r-- 1 jhanasri jhanasri    2092 Aug 17 05:03 task5_process.c
-rw-r--r-- 1 jhanasri jhanasri    2131 Sep 21 06:09 task_signal_forward.c.save
-rw-r--r-- 1 jhanasri jhanasri    2234 Sep 21 06:01 task_resume_job.c
-rw-r--r-- 1 jhanasri jhanasri    2300 Aug 13 11:56 task2_quotes.c
-rw-r--r-- 1 jhanasri jhanasri    2450 Sep 21 05:39 task_process_groups.c
-rw-r--r-- 1 jhanasri jhanasri    2602 Sep 21 05:29 task_sigstp.c
-rw-r--r-- 1 jhanasri jhanasri    2612 Aug  7 08:30 task5_memory.c
-rw-r--r-- 1 jhanasri jhanasri    2705 Sep 22 14:27 task_large_pipeline.c
-rw-r--r-- 1 jhanasri jhanasri    3056 Sep 18 04:11 week_5_double_quotes.c
-rw-r--r-- 1 jhanasri jhanasri    3056 Sep 18 04:38 task3_double_quotes.c
-rw-r--r-- 1 jhanasri jhanasri    3976 Aug  7 08:19 task4_history.c
-rwxr-xr-x 1 jhanasri jhanasri    1107 Sep 22 14:16 test_script.sh
-rwxr-xr-x 1 jhanasri jhanasri   16064 Sep 22 14:14 calculator_test
-rwxr-xr-x 1 jhanasri jhanasri   16360 Sep 21 04:56 task_signal_actions
-rwxr-xr-x 1 jhanasri jhanasri   16392 Sep 18 03:51 client
-rwxr-xr-x 1 jhanasri jhanasri   16400 Aug 17 04:50 task4_escape
-rwxr-xr-x 1 jhanasri jhanasri   16400 Sep 18 03:26 producer_consumer
-rwxr-xr-x 1 jhanasri jhanasri   16648 Sep 21 05:39 task_process_groups
-rwxr-xr-x 1 jhanasri jhanasri   16696 Aug 17 05:03 task5_process
drwxr-xr-x 2 jhanasri jhanasri    4096 Aug  1 09:07 include

```

```

drwxr-xr-x 2 jhanasri jhanasri 4096 Aug 1 09:07 include
drwxr-xr-x 2 jhanasri jhanasri 4096 Aug 1 09:08 docs
drwxr-xr-x 2 jhanasri jhanasri 4096 Aug 1 09:08 scripts
drwxr-xr-x 2 jhanasri jhanasri 4096 Aug 4 05:32 src
prw-r--r-- 1 jhanasri jhanasri 0 Sep 18 03:52 client_to_server
prw-r--r-- 1 jhanasri jhanasri 0 Sep 18 03:52 server_to_client

```

Pipeline execution completed.

Job 1: SUCCESS

Job 2: SUCCESS

Job 3: SUCCESS

Total execution time: 0.061923 seconds

jhanasri jhanasri:~/OSSP\$ ps -ef | grep task_large_pipeline

jhanasri 716 677 0 14:27 pts/0 00:00:00 grep --color=auto task_large_pipeline

jhanasri jhanasri:~/OSSP\$ top

top - 14:27:46 up 48 min, 1 user, load average: 0.06, 0.04, 0.00

Tasks: 22 total, 1 running, 21 sleeping, 0 stopped, 0 zombie

%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 99.8 id, 0.2 wa, 0.0 hi, 0.0 si, 0.0 st

Mem : 15743.6 total, 15097.3 free, 590.8 used, 251.4 buff/cache

Swap: 4096.0 total, 4096.0 free, 0.0 used, 15152.8 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	21736	13228	9768	S	0.0	0.1	0:00.38	systemd
2	root	20	0	3180	2208	2072	S	0.0	0.0	0:00.00	init-systemd(Ub
6	root	20	0	3196	2132	2008	S	0.0	0.0	0:00.00	init
49	root	19	-1	50440	16248	15160	S	0.0	0.1	0:00.17	systemd-journal
101	root	20	0	25624	6752	5136	S	0.0	0.0	0:00.57	systemd-udev
114	systemd+	20	0	21344	13156	11032	S	0.0	0.1	0:00.05	systemd-resolve
115	systemd+	20	0	91036	7964	6996	S	0.0	0.0	0:00.05	systemd-timesyn
164	root	20	0	4244	2688	2432	S	0.0	0.0	0:00.00	cron
168	message+	20	0	9648	5412	4700	S	0.0	0.0	0:00.03	dbus-daemon
176	root	20	0	17976	8792	7748	S	0.0	0.1	0:00.03	systemd-logind
195	syslog	20	0	222516	5636	4592	S	0.0	0.0	0:00.04	rsyslogd
198	root	20	0	3124	1960	1824	S	0.0	0.0	0:00.00	agetty
199	root	20	0	107036	23196	13744	S	0.0	0.1	0:00.07	unattended-upgr
288	root	20	0	6696	4692	3916	S	0.0	0.0	0:00.01	login
331	jhanasri	20	0	20320	11512	9392	S	0.0	0.1	0:00.04	systemd
332	jhanasri	20	0	21168	3596	1868	S	0.0	0.0	0:00.00	(sd-pam)
351	jhanasri	20	0	6888	5396	3708	S	0.0	0.0	0:00.01	bash
633	root	20	0	1756628	13704	11448	S	0.0	0.1	0:00.12	wsl-pro-service
673	root	20	0	3184	1120	988	S	0.0	0.0	0:00.00	SessionLeader
675	root	20	0	3200	1256	1108	S	0.0	0.0	0:00.28	Relay(677)
677	jhanasri	20	0	6208	5408	3652	S	0.0	0.0	0:00.02	bash
717	jhanasri	20	0	9392	5820	4712	R	0.0	0.0	0:00.01	top


```

jhanasri jhanasri:~/OSSP$ cd -/OSSP
jhanasri jhanasri:~/OSSP$ mamo task_large_pipeline.c
jhanasri jhanasri:~/OSSP$ get -rdr -y jhanasri task_large_pipeline
(anga pipeline Hascadion -/"task_large_pipeline
=====
There pipeline jobs launched.
Job 1: t8 -/
job 2: grem .c
Job 3: veit

-rw-r--r-- 1 jhanasri jhanasri 77 Sep 22 14:17 expected_report.txt
-rw-r--r-- 1 jhanasri jhanasri 130 Sep 22 13:39 calculator.h
-rw-r--r-- 1 jhanasri jhanasri 169 Sep 22 13:41 calculator.c
-rw-r--r-- 1 jhanasri jhanasri 355 Sep 22 14:14 calculator_test.c
-rw-r--r-- 1 jhanasri jhanasri 420 Aug 13 11:45 main_loop.c
-rw-r--r-- 1 jhanasri jhanasri 443 Sep 22 14:07 task_valgrind_leak.c
-rw-r--r-- 1 jhanasri jhanasri 465 Sep 22 13:42 main_module.c
-rw-r--r-- 1 jhanasri jhanasri 662 Aug 17 13:53 task11_T2.c
-rw-r--r-- 1 jhanasri jhanasri 726 Sep 18 03:58 signal_handler.c
-rw-r--r-- 1 jhanasri jhanasri 769 Sep 18 14:03 task_valgrind.c
-rw-r--r-- 1 jhanasri jhanasri 900 Aug 17 06:02 task3_fort.c
-rw-r--r-- 1 jhanasri jhanasri 954 Aug 17 13:32 task4_states.c
-rw-r--r-- 1 jhanasri jhanasri 1018 Sep 18 03:50 client.c
-rw-r--r-- 1 jhanasri jhanasri 1275 Sep 18 03:48 server.c
-rw-r--r-- 1 jhanasri jhanasri 1376 Sep 18 03:32 ls_grep.c
-rw-r--r-- 1 jhanasri jhanasri 1464 Sep 22 13:44 calculator.o
-rw-r--r-- 1 jhanasri jhanasri 1541 Sep 22 13:50 task_error_handling.c
-rw-r--r-- 1 jhanasri jhanasri 1550 Sep 21 04:56 task_signal_actions.c
-rw-r--r-- 1 jhanasri jhanasri 1617 Sep 21 05:54 task_signal_behavior.c
-rw-r--r-- 1 jhanasri jhanasri 1661 Aug 7 08:09 task3_input.c
-rw-r--r-- 1 jhanasri jhanasri 1677 Sep 18 03:26 producer_consumer.c
-rw-r--r-- 1 jhanasri jhanasri 1705 Sep 18 04:13 task2_single_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 1851 Aug 17 13:42 task1_wait.c
-rw-r--r-- 1 jhanasri jhanasri 1851 Aug 17 13:46 task1_wait.c.save
-rw-r--r-- 1 jhanasri jhanasri 1959 Sep 21 05:15 task_signit_forward.c
-rw-r--r-- 1 jhanasri jhanasri 2092 Aug 17 05:03 task5_process.c
-rw-r--r-- 1 jhanasri jhanasri 2131 Sep 21 05:09 task_sigint_forward.c
-rw-r--r-- 1 jhanasri jhanasri 2234 Sep 21 06:01 task_resume_job.c
-rw-r--r-- 1 jhanasri jhanasri 2297 Aug 17 04:59 task4_escape.c
-rw-r--r-- 1 jhanasri jhanasri 2380 Aug 13 11:56 task2_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 2450 Sep 21 05:39 task_process_groups.c
-rw-r--r-- 1 jhanasri jhanasri 2602 Sep 21 05:29 task_sigstp.c
-rw-r--r-- 1 jhanasri jhanasri 2612 Aug 7 08:30 task5_memory.c
-rw-r--r-- 1 jhanasri jhanasri 2705 Sep 22 14:27 task_large_pipeline.c
-rw-r--r-- 1 jhanasri jhanasri 3096 Sep 18 04:11 week_5_double_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 3056 Sep 18 04:38 task3_double_quotes.c
-rw-r--r-- 1 jhanasri jhanasri 3976 Aug 7 08:19 task4_history.c

```

Observation:

The program created multiple child processes and executed pipeline commands successfully. The execution of different jobs was monitored, system resources were observed, and the overall execution time was measured to check system stability and performance.

Result:

The large pipeline executed successfully, demonstrating process creation, pipeline execution, job monitoring, failure detection, and performance analysis.

2 . To document the system architecture, create flow diagrams, explain execution paths, describe system calls, record design decisions, and review the project documentation.

Aim : To document the architecture and execution flow of the system, identify the system calls used, record important design decisions, and maintain clear and organized project documentation.

Program :

```
GNU nano 7.2
SYSTEM ARCHITECTURE

1. User Interface
  |
  v
2. Command Parser
  |
  v
3. Command Executor
  |
  v
4. Process Management
  |
  v
5. Linux Kernel
  |
  v
6. Child Process / Program

Main Components:
- Parser: Reads and analyzes commands.
- Executor: Executes requested programs.
- Process Manager: Creates and manages processes.
- Signal Handler: Handles process signals.
- Linux Kernel: Provides system-call services.
```

EXECUTION FLOW

Start

|

v

Read Command

|

v

Validate Command

|

v

Parse Arguments

|

v

Create Child Process using fork()

|

v

Execute Program using exec()

|

v

Parent waits using wait()/waitpid()

|

v

Check Exit Status

|

v

Display Result

|

v

End


```
GNU nano 7.2
SYSTEM CALLS

fork()
- Creates a new child process.

exec()
- Replaces the child process with a new program.

wait()
- Waits for a child process to finish.

waitpid()
- Waits for a specific child process.

pipe()
- Creates communication between processes.

dup2()
- Redirects file descriptors.

kill()
- Sends a signal to a process.

signal()
- Registers a signal handler.

sigaction()
- Configures signal handling.

open()
- Opens a file.

close()
- Closes a file descriptor.
```

```
GNU nano 7.2 docs/
DESIGN DECISIONS

1. Use modular C files for better organization.
2. Use fork() to create child processes.
3. Use exec() to execute external programs.
4. Use waitpid() to synchronize parent and child processes.
5. Use pipes for inter-process communication.
6. Use signal handling for process control.
7. Use separate documentation files for architecture,
   execution flow, system calls, and design decisions.
8. Use compiler warnings during compilation to improve reliability.
```

Output :

```
jhanasri jhanasri:~/OSSP$ cd ~/OSSP
jhanasri jhanasri:~/OSSP$ mkdir -p docs
jhanasri jhanasri:~/OSSP$ nano docs/architecture.txt
jhanasri jhanasri:~/OSSP$ nano docs/execution_flow.txt
jhanasri jhanasri:~/OSSP$ nano docs/syscalls.txt
jhanasri jhanasri:~/OSSP$ nano docs/design_decisions.txt
jhanasri jhanasri:~/OSSP$ cat docs/architecture.txt
```

SYSTEM ARCHITECTURE

1. User Interface
↓
2. Command Parser
↓
3. Command Executor
↓
4. Process Management
↓
5. Linux Kernel
↓
6. Child Process / Program

Main Components:

- Parser: Reads and analyzes commands.
- Executor: Executes requested programs.
- Process Manager: Creates and manages processes.
- Signal Handler: Handles process signals.
- Linux Kernel: Provides system-call services.

```
jhanasri jhanasri:~/OSSP$ cat docs/execution_flow.txt
```

EXECUTION FLOW

```
Start
↓
Read Command
↓
Validate Command
↓
Parse Arguments
↓
Create Child Process using fork()
↓
Execute Program using exec()
↓
Parent waits using wait()/waitpid()
↓
Check Exit Status
↓
Display Result
↓
End
```

```
jhanasri jhanasri:~/OSSP$ cat docs/syscalls.txt  
SYSTEM CALLS
```

```
fork()
```

```
- Creates a new child process.
```

```
exec()
```

```
- Replaces the child process with a new program.
```

```
wait()
```

```
- Waits for a child process to finish.
```

```
waitpid()
```

```
- Waits for a specific child process.
```

```
pipe()
```

```
- Creates communication between processes.
```

```
dup2()
```

```
- Redirects file descriptors.
```

```
kill()
```

```
- Sends a signal to a process.
```

```
signal()
```

```
- Registers a signal handler.
```

```
sigaction()
```

```
- Configures signal handling.
```

```
open()
```

```
- Opens a file.
```

```
close()
```

```
- Closes a file descriptor.
```

```
jhanasri jhanasri:~/OSSP$ cat docs/design_decisions.txt  
DESIGN DECISIONS
```

1. Use modular C files for better organization.
2. Use fork() to create child processes.
3. Use exec() to execute external programs.
4. Use waitpid() to synchronize parent and child processes.
5. Use pipes for inter-process communication.
6. Use signal handling for process control.
7. Use separate documentation files for architecture, execution flow, system calls, and design decisions.
8. Use compiler warnings during compilation to improve reliability.

```
jhanasri jhanasri:~/OSSP$ tree docs
docs
├── architecture.txt
├── design.md
├── design_decisions.txt
├── execution_flow.txt
├── report.pdf
└── syscalls.txt

1 directory, 6 files
jhanasri jhanasri:~/OSSP$ |
```

Observation:

The project documentation was organized into architecture, execution flow, system calls, and design decisions. The documented files clearly described the system structure, command execution process, important system calls, and design choices.

Result:

The project was successfully documented with a clear architecture, execution flow, system-call descriptions, and design decisions, making the system easier to understand and review.

3 . To organize source code, improve build scripts, add a README file, verify repository structure, tag releases, and validate project deliverables.

Aim : To maintain a well-organized project repository with a reliable build process, clear documentation, proper version tags, and verified project deliverables.

Program :

```

GNU nano 7.2                                     READ
# OSSIP Project

## Description
This project demonstrates Operating System and Systems Programming concepts using C.

## Project Structure
- src/      - Source code
- include/  - Header files
- obj/      - Object files
- bin/      - Executable files
- docs/     - Documentation
- tests/    - Test programs
- scripts/  - Shell scripts

## Build
Use:
make

## Run
Use:
./bin/my_shell

## Clean
Use:
make clean

## Testing
Test programs are stored in the tests directory.

## Version
v1.0.0

```

```

GNU nano 7.2
CC = gcc
CFLAGS = -Wall -Wextra -Iinclude

TARGET = bin/my_shell

SOURCES = src/main.c \
          src/shell.c \
          src/parser.c \
          src/executor.c \
          src/builtin.c

OBJECTS = $(SOURCES:src/%.c=obj/%.o)

all: $(TARGET)

$(TARGET): $(OBJECTS)
    mkdir -p bin
    $(CC) $(OBJECTS) -o $(TARGET)

obj/%.o: src/%.c
    mkdir -p obj
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -rf obj/*.o bin/my_shell

test: $(TARGET)
    @echo "Running project tests..."
    @echo "Tests completed."

.PHONY: all clean test

```

Output :



































Observation:

The source code, header files, documentation, scripts, and build files were organized into the appropriate project directories. The repository structure, build process, Git status, commits, and release tag were checked to verify the project deliverables.

Result:

The project repository was successfully organized and validated. The project was committed and tagged with the required release information, and the repository structure was verified for final submission.

4 . To create demo scenarios, validate project features, rehearse the presentation, demonstrate pipelines, demonstrate signal handling, and perform the final project review.

Aim : To verify the complete project functionality through practical demonstrations, test important features such as pipelines and signals, and prepare the project for final presentation and evaluation.

Output :

TERMINAL - 1 :









TERMINAL - 2 :











**Observation:**

The project was tested through practical demonstrations using two terminal sessions. Pipeline execution, process management, signal handling, and other important project features were executed and verified during the final review.

Result:

The major project features were successfully demonstrated and validated. The final review confirmed the working functionality of the project and prepared it for final presentation and evaluation.